

PSTAT 10 Data Science Principles

Lecture 7: Tibbles

John Robin Inston 

johninston@ucsb.edu

University of California, Santa Barbara

August 12, 2026

Introduction

Review: Lecture 6

👉 Last lecture we studied the following topics:

- **Real Data**
 - Loading Data
 - Summarizing Data
 - Visualizing Data
- **Exploratory Data Analysis**
 - Quantitative vs Categorical
 - Factor Data
 - Histograms

👁️👁️ Outline: Lecture 7

👉 Today we will move on to looking at **Tibble** objects and functions from the `tidyverse` library, specifically:

- Issues with dataframes
- **Tibble Objects**
- The **pipe operator** `|>`
- `dplyr` operators from the `tidyverse` package
- **Reading external data**

```
as_tibble(iris)
```

A tibble: 150 × 5

Sepal.Length <dbl>	Sepal.Width <dbl>	Petal.Length <dbl>	Petal.Width <dbl>	Species <fctr>
5.1	3.5	1.4	0.2	setosa
4.9	3.0	1.4	0.2	setosa
4.7	3.2	1.3	0.2	setosa
4.6	3.1	1.5	0.2	setosa
5.0	3.6	1.4	0.2	setosa
5.4	3.9	1.7	0.4	setosa
4.6	3.4	1.4	0.3	setosa
5.0	3.4	1.5	0.2	setosa
4.4	2.9	1.4	0.2	setosa
4.9	3.1	1.5	0.1	setosa

1-10 of 150 rows

Previous 2 3 4 5 6 ... 15 Next

Tibbles

Tibble Objects

! Problems with Dataframes

We have seen that **dataframes** are helpful but come with some frustrating drawbacks:

- They **allow numerical operations to run on non-numerical data**, producing **NA** values.

```
1 iris$Species[c(1,70,150)]
```

```
[1] setosa    versicolor virginica  
Levels: setosa versicolor virginica
```

```
1 iris$Species[c(1,70,150)]^2
```

```
[1] NA NA NA
```

- **No sequential column evaluation.**

```
1 broken_df <- data.frame(x = 1:5, y = x^2)
```

```
Error in eval(expr, envir, enclos): object 'x' not found
```

- **Bad list compatibility.**

```
1 list_df <- data.frame(x = 1:3, y = list(1:5, 1:10, 1:20))
```

```
Error in data.frame(x = 1:3, y = list(1:5, 1:10, 1:20)): arguments  
imply differing number of rows: 3, 20
```

- Somewhat **ambiguous subsetting.**

```
1 is.data.frame(iris[1, ])
```

```
[1] TRUE
```

```
1 is.data.frame(iris[,1])
```

```
[1] FALSE
```

- **Partial matching with `$`.**

```
1 partial_df <- data.frame(abc = 1, def = 2)  
2 partial_df$d
```

```
[1] 2
```

Tibble Objects

The `tidyverse` package provides a solution in the form of `tibble()` objects.

First, make sure you have the `tidyverse` library loaded:

```
1 library(tidyverse)
```

We can use the `tibble()` function to convert a dataframe into a tibble.

```
1 tibble(  
2   ...,  
3   .rows = NULL,  
4   .name_repair = c("check_unique", "unique", "universal", "minimal")  
5 )
```

 There are lots of helpful arguments (such as `.name_repair` above) allowing us to modify dataframes to be easier to manipulate — look them up with `?tibble()`.

```
1 cabbages_tib <- tibble(MASS::cabbages); cabbages_tib
```

```
# A tibble: 60 × 4  
  Cult Date HeadWt VitC  
  <fct> <fct>   <dbl> <int>  
1 c39 d16      2.5    51  
2 c39 d16      2.2    55  
3 c39 d16      3.1    45  
4 c39 d16      4.3    42  
5 c39 d16      2.5    53  
6 c39 d16      4.3    50  
7 c39 d16      3.8    50  
8 c39 d16      4.3    52  
9 c39 d16      1.7    56  
10 c39 d16      3.1    49  
# i 50 more rows
```



Tibble Benefits

Sequential Column Evaluation

Tibbles **define columns sequentially**:

```
1 (seq_tib <- tibble(x = 1:3, y = x^2))
```

```
# A tibble: 3 × 2
```

	x	y
	<int>	<dbl>
1	1	1
2	2	4
3	3	9

Subsetting & Matching

Subsetting a tibble always **returns a tibble**, and tibbles **do not allow partial matching**:

```
1 is_tibble(cabbages_tib[1,])
```

```
[1] TRUE
```

```
1 cabbages_tib$HeadW
```

```
NULL
```

Arithmetic

Tibbles **do not support arithmetic across columns**:

```
1 cabbages_tib[1:4, ] * 2
```

	Cult	Date	HeadWt	VitC
1	NA	NA	5.0	102
2	NA	NA	4.4	110
3	NA	NA	6.2	90
4	NA	NA	8.6	84

Recycling

Tibble **recycling is extremely strict**:

```
1 tibble(a = 1, b = 1:3)
```

```
# A tibble: 3 × 2
```

	a	b
	<dbl>	<int>
1	1	1
2	1	2
3	1	3

```
1 tibble(a = 1:2, b = 1:6)
```

```
Error in `tibble()`:
! Tibble columns must have compatible sizes.
• Size 2: Existing data.
• Size 6: Column `b`.
i Only values of size one are recycled.
```


Pipe Operator

Functional Composition

We often wish to **apply one function after another after another** when performing computations, a process known as **functional composition**.

Example: Say we wish to find which (numeric) column in the `iris` dataset has the highest mean.

```
1 # manually composing functions
2 which.max(apply(iris[,-5], 2, mean))
```

```
Sepal.Length
1
```

 Here we first used the function `mean()`, which lies inside `apply()`, which is **nested** inside the function `which.max()` to get the desired result.

The Pipe Operator

Nesting functions works fine but can quickly become very **difficult to read**.

The **pipe operator** `|>` allows us to **chain (compose) several functions together** in a clear and easy to read way. The pipe operator passes the **left-hand side item** as the **first argument to the right-hand side function** and returns the result.

```
1 # compute sepal length mean
2 iris$Sepal.Length |> mean()
```

```
[1] 5.843333
```

Pipe Operator Chains

Start chaining functions!

- This is by no means limited to being applied once!
 - The pipe operator allows us to construct **long chains of functions** where the output of the first is fed into the second and so on.

Example: We can rewrite our previous function with pipe operators.

```
1 # composition with pipe function
2 iris[-5] |>
3   apply(2, mean) |>
4   which.max()
```

```
Sepal.Length
1
```

 The pipe operator is particularly helpful when we begin manipulating tibble objects using the `dplyr` functions in the following section.

- Note here that we typically start a new line following every pipe operator to make the code more readable.

Exercise — Pipe Operator

Consider the following numerical vector:

```
1 x <- c(0.109, 0.359, 0.63, 0.996, 0.515, 0.142, 0.017, 0.829, 0.907)
```

The following nested function:

1. Computes the natural log: `log(...)`
2. Computes the difference with lag 1: `diff(..., lag = 1)`
3. Computes the exponential: `exp(...)`
4. Rounds the solution to the nearest 1 d.p.: `round(..., digits = 1)`

```
1 round(exp(diff(log(x))), 1)
```

```
[1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

Write the equivalent function using the pipe `|>` operator.

03:00

✓ Solution — Pipe Operator

We **chain** `log()`, `diff()`, `exp()` and `round()` together using the pipe operator:

```
1 x |>  
2   log() |>  
3   diff(1) |>  
4   exp() |>  
5   round(1)
```

```
[1] 3.3 1.8 1.6 0.5 0.3 0.1 48.8 1.1
```

📌 If we are calling a function but the pipe has **already defined the first input**, we still need the parentheses but leave them blank.

As a full function we could write:

```
1 piped_function <- function(x){  
2   x |>  
3   log() |>  
4   diff(1) |>  
5   exp() |>  
6   round(1)  
7 }
```

Tibble Manipulation

dplyr Functions

Working with Tibbles

The `dplyr` functions are designed to **interface easily** with tibble data. The functions are loaded via the `tidyverse` package.

The functions are:

- `select()` — **select specific columns**;
- `filter()` — **filter rows** based on logical conditions;
- `mutate()` — **define new columns** or **redefine existing columns**;
- `summarise()` — creates a **new data frame of summary statistics** of data grouped by a chosen variable; and
- `arrange()` — **orders tibble rows** based on a selected column.

These are often referred to as the “verbs” of `dplyr` because they describe the action we are taking on the data.

`select()` Function

The `select()` function allows us to **choose specific columns of a tibble object** using either column names or indices.

By Name

Select using column names:

```
1 # define data and make tibble
2 cabbages <- MASS::cabbages |> tibble()
3 # select the Cult, Date and HeadWt columns
4 cabbages |> select(Cult, Date, HeadWt) |> head(2)
```

```
# A tibble: 2 × 3
  Cult   Date HeadWt
<fct> <fct>   <dbl>
1 c39   d16     2.5
2 c39   d16     2.2
```

By Index

Select using column index numbers:

```
1 # select the columns 1 to 3
2 cabbages |>
3   select(1:3) |>
4   head(2)
```

```
# A tibble: 2 × 3
  Cult   Date HeadWt
<fct> <fct>   <dbl>
1 c39   d16     2.5
2 c39   d16     2.2
```


`select()` — Deselecting & Patterns

Deselecting

Select which columns to remove:

```
1 # deselect VitC column
2 cabbages |>
3   select(-VitC) |>
4   head(2)
```

```
# A tibble: 2 × 3
  Cult   Date HeadWt
<fct> <fct>   <dbl>
1 c39   d16     2.5
2 c39   d16     2.2
```

By Name Pattern

Select all columns whose names **share a common pattern** using helpers like `starts_with()`:

```
1 iris |>
2   tibble() |>
3   select(starts_with("Sepal")) |>
4   head(2)
```

```
# A tibble: 2 × 2
  Sepal.Length Sepal.Width
      <dbl>         <dbl>
1         5.1         3.5
2         4.9         3
```

`filter()` Function

The `filter()` function allows us to filter rows based on logical conditions satisfied by each row.

Single Condition

Return rows that have `Date` of either `"d20"` or `"d21"`:

```
1 cabbages |>
2   filter(Date == "d20" | Date == "d21") |>
3   head(2)
```

```
# A tibble: 2 × 4
  Cult Date  HeadWt VitC
<fct> <fct>   <dbl> <int>
1 c39 d20      3     65
2 c39 d20     2.8     52
```

Multiple Conditions

Return rows with (i) `Cult` of `"c39"`, (ii) `Date` of `"d16"`, and (iii) `HeadWt` between 2.8 and 3.3:

```
1 cabbages |>
2   filter(
3     Cult == "c39" & Date == "d16" &
4     HeadWt > 2.8 & HeadWt < 3.3
5   ) |>
6   head(2)
```

```
# A tibble: 2 × 4
  Cult Date  HeadWt VitC
<fct> <fct>   <dbl> <int>
1 c39 d16      3.1     45
2 c39 d16      3.1     49
```



Exercise — `filter()` and `select()`

We shall use the `painters` dataframe from the `MASS` package, loaded into our environment using the following code:

```
1 painters <- MASS::painters
```

1. Convert the `painters` dataframe into a tibble and save it to a new object called `painters_tib`.
2. Use the `select()` and `filter()` functions and the pipe operator `|>` to obtain a tibble object with columns for `School`, `Drawing` and `Expression` containing all painters from `School A` with Drawing and Expression scores **above 5**.

05:00

✓ Solution — `filter()` and `select()`

We **convert** `painters` to a tibble, then use `select()` and `filter()` to obtain the required columns and rows:

```
1 # convert to tibble
2 painters_tib <- painters |> tibble()
3 # obtain required painters
4 painters_tib |>
5   select(School, Drawing, Expression) |>
6   filter(School == "A" & Drawing > 5 & Expression > 5)
```

```
# A tibble: 7 × 3
  School Drawing Expression
  <fct>    <int>    <int>
1 A         16         14
2 A         13          7
3 A         16          8
4 A         16         14
5 A         17          8
6 A         16          6
7 A         18         18
```



mutate() Function

New Columns

Take the `cabbages` tibble we defined earlier. If we want to add a new column named `Color` we can do so using the `mutate()` function:

```
1 cabbages |>
2   mutate(Color = rep("Green", length(Cult))) |>
3   head(2)
```

```
# A tibble: 2 × 5
  Cult Date   HeadWt VitC Color
<fct> <fct>   <dbl> <int> <chr>
1 c39  d16     2.5    51 Green
2 c39  d16     2.2    55 Green
```

New Columns from Existing

Alternatively we can define a new column using data from existing columns. For example, we can define a new column called `VitCPer` detailing the `VitC` per `HeadWt`.

```
1 cabbages |>
2   mutate(VitCPer = VitC / HeadWt) |>
3   head(2)
```

```
# A tibble: 2 × 5
  Cult Date   HeadWt VitC VitCPer
<fct> <fct>   <dbl> <int>   <dbl>
1 c39  d16     2.5    51    20.4
2 c39  d16     2.2    55     25
```



`mutate()` — Changing & Removing Columns

Changing Existing Columns

`mutate()` can also be used to change existing columns by simply stating the name of the column you wish to change.

For example, say we discover that due to equipment failure, the `HeadWt` variable should actually be 1.6 times bigger for all observations.

```
1 cabbages |> mutate(HeadWt = 1.6 * HeadWt) |> head(2)
```

```
# A tibble: 2 × 4  
  Cult Date HeadWt VitC  
  <fct> <fct>   <dbl> <int>  
1 c39  d16      4      51  
2 c39  d16     3.52     55
```

Removing a Column

We can also **remove a column** entirely by setting it to `NULL`:

```
1 cabbages |>  
2   mutate(HeadWt = NULL) |>  
3   head(2)
```

```
# A tibble: 2 × 3  
  Cult Date VitC  
  <fct> <fct> <int>  
1 c39  d16     51  
2 c39  d16     55
```

Reading Data

Reading External Data

Lots of file types!

- One problem with reading data into R is that data can come in a variety of **different formats** and **file types** (some easy to use, some challenging).
- R has lots of functionality for dealing with these subtleties, but most problems can be reduced by checking a few key things:
 - Is the file you are trying to load **in your working directory** (i.e. the same directory as your R console and your Quarto document)?
 - Have I written the **file path of the data file** correctly (spelling, file extension, etc.)?
 - Did I use the **correct function for the file type**?

Some good practices

- Create a folder for each Quarto document you are working on.
- Create a sub-folder called `data` in this folder to store all your data files.
- Make sure to **check your working directory** and **file paths** before trying to load data files into R.

Downloading Data Files

Data on Canvas

- On Canvas you should be able to see a module named `Data`.
 - This is where you will find the data files for this course.
- For this lecture we will be using the following three files:
 1. `class.txt`
 2. `customers-100.csv`
 3. `SampleData.xlsx`

If you are working along, save all three files to the `data` folder within your current working directory.

`.txt` Files

What are `.txt` Files?

- A `.txt` file is a **plain text file** that can be opened in any text editor (e.g. Notepad, TextEdit, etc.) and contains data in a tabular format.
- To load `.txt` files we use the function `read.table()`:

```
1 class <- read.table("data/class.txt",  
2                     header = TRUE,  
3                     sep = " ")  
4 head(class)
```

	Name	Sex	Age	Height	Weight
1	Alfred	M	14	69.0	112.5
2	Alice	F	13	56.5	84.0
3	Barbara	F	13	65.3	98.0
4	Carol	F	14	62.8	102.5
5	Henry	M	14	63.5	102.5
6	James	M	12	57.3	83.0

Reading `.txt` Files — Key Arguments

- In this case we noticed that the `.txt` file had a header, so we specified `header = TRUE`.
- We also noticed that the elements of the data were **separated by spaces**, so we specified `sep = " "`.

`.csv` Files

What are `.csv` Files?

- A `.csv` file is a **comma-separated value file** (similar to an Excel spreadsheet but with fewer formatting options) that stores tabular data as plain text, with commas separating each value.
- To load `.csv` files we use the function `read.csv()`:

```
1 customers <- read.csv("data/customers-100.csv")
2 head(customers[,1:4])
```

	Index	Customer.Id	First.Name	Last.Name
1	1	DD37Cf93aecA6Dc	Sheryl	Baxter
2	2	1Ef7b82A4CAAD10	Preston	Lozano
3	3	6F94879bDAfE5a6	Roy	Berry
4	4	5Cef8BFA16c5e3c	Linda	Olsen
5	5	053d585Ab6b3159	Joanna	Bender
6	6	2d08FB17EE273F4	Aimee	Downs

Reading `.csv` Files — Key Arguments

- `read.csv()` assumes the file has a **header row** and is **comma-separated** by default, so unlike `read.table()` we don't need to specify `header` or `sep` arguments.

.xlsx Files

What are .xlsx Files?

- An `.xlsx` file is an **Excel spreadsheet** that may contain multiple sheets of tabular data.
- To load `.xlsx` files we use the function `read_excel()` from the `readxl` package:

```
1 SampleData <- readxl::read_excel(  
2   "data/SampleData.xlsx",  
3   sheet = 2  
4 )  
5 head(SampleData)
```

```
# A tibble: 6 × 7  
  OrderDate      Region Rep    Item  Units `Unit Cost` Total  
  <dtm>        <chr> <chr> <chr> <dbl>    <dbl> <dbl>  
1 2021-01-06 00:00:00 East   Jones Pencil    95      1.99  189.  
2 2021-01-23 00:00:00 Central Kivell Binder    50     20.0  999.  
3 2021-02-09 00:00:00 Central Jardine Pencil    36      4.99  180.  
4 2021-02-26 00:00:00 Central Gill    Pen     27     20.0  540.  
5 2021-03-15 00:00:00 West   Sorvino Pencil    56      2.99  167.  
6 2021-04-01 00:00:00 East   Jones Binder    60      4.99  299.
```

Reading .xlsx Files — Key Arguments

- Excel workbooks can contain **multiple sheets**, so we specify `sheet = 2` to tell `read_excel()` which sheet to read from.

Exercise — Working with External Data

You will find on Canvas the dataset `faithful.csv`, which you will need for this final exercise.

1. Download `faithful.csv` and put it in the correct directory.
2. Load the data into R, storing it in an object called `faithful` using the function `read.csv()`.
3. Convert `faithful` to a tibble.
4. Create a new column called `scaled` which contains the eruptions divided by waiting time of Old Faithful.
5. Display only the resulting rows corresponding to eruption duration of 5 or more.

06:00

Solution — Working with External Data

We **load** the data, convert it to a tibble, then use `mutate()` and `filter()` to compute the scaled column and select the required rows:

```
1 # load data
2 faithful <- read.csv("data/faithful.csv")
3 # display required rows
4 faithful |>
5   tibble() |>
6   mutate(scaled = eruptions / waiting ) |>
7   filter(eruptions > 5)
```

```
# A tibble: 3 × 4
  X eruptions waiting scaled
<int>    <dbl>    <int>  <dbl>
1    76     5.07     76 0.0667
2   149     5.1      96 0.0531
3   151     5.03     77 0.0654
```

Summary

Topics Covered

 Today we looked into:

- Tibble Objects
- Pipe Operator
- Tibble Manipulation
 - Selecting
 - Filtering
 - Mutating
- Reading Data
 - `.txt`
 - `.csv`
 - `.xlsx`



Next Class



Next class we will study:

- Descriptive statistics
- Base R Plotting
- Probability
- Discrete and Continuous Random Variables